



Lokale KI-Infrastruktur

Technischer Bericht für den KI Coach

Erstellt von Sven

Datum 25. April 2026

Frontend Open WebUI

Backend Ollama (lokal auf macOS)

Modelle 2 KI-Modelle + 1 WebUI-Feature

Was läuft bei mir lokal?

Auf meinem lokalen Rechner (macOS) betreibe ich eine vollständige KI-Infrastruktur bestehend aus **Ollama** als Modell-Backend und **Open WebUI** als Browser-basierter Benutzeroberfläche. Aktuell sind zwei Sprachmodelle installiert, beide basierend auf **Meta Llama 3.1 8B**.

Alle Anfragen bleiben auf meinem Gerät – keine Daten verlassen meinen Rechner. Der Betrieb ist kostenlos und funktioniert offline.


```
Dein Mac (Host / "lokal")
```

```
| | | Open  
WebUI (Browser-Frontend) | | → Zugriff über: http://localhost:3000 | |  
| | Ollama (KI-Backend / Modell-Manager) | | → API unter:  
http://localhost:11434 | | | Modelle (Dateien unter  
~/.ollama/models/) | | └─ sven-llama3.1:8b-instruct-q5_K_M (~5.3 GB) |  
|   └─ llama3.1:8b-instruct-q5_K_M (~5.3 GB) | | |  
|
```

KOMPONENTE	SOFTWARE	FUNKTION	PORT
Frontend	Open WebUI	Browser-Oberfläche (Chat, Modellwahl, Einstellungen)	3000

Backend	Ollama	Lädt und führt KI-Modelle aus, stellt API bereit	11434
Modell 1	sven-llama3.1	Persönlich angepasstes Modell mit eigenem Systemprompt	–
Modell 2	llama3.1	Standard-Modell, flexibel nutzbar	–
Feature	Arena Model	Blindvergleich zwischen Modellen (A/B-Test)	–

Installierte Modelle im Detail

sven-llama3.1:8b-instruct-q5_K_M

CUSTOM

TECHNISCHE DATEN

Basis-Modell	Meta Llama 3.1 8B Instruct
Quantisierung	Q5_K_M (5.7 Bit/Gewicht)
Speicherbedarf	~5.3 GB RAM/VRAM
Kontextfenster	128.000 Token
Erstellt mit	<code>ollama create</code> aus Modelfile
Besonderheit	Dauerhafter persönlicher Systemprompt

Was macht dieses Modell besonders?

Das Präfix `sven-` zeigt, dass ich dieses Modell selbst aus dem Standard-Llama 3.1 erstellt und mit einem persönlichen Systemprompt konfiguriert habe. Der Systemprompt ist dauerhaft im Modell eingebettet – ich muss ihn bei jedem Chat nicht neu eingeben.

Systemprompt einsehen (Terminal-Befehl):

```
ollama show sven-llama3.1:8b-instruct-q5_K_M --modelfile
```

API-Aufruf:

```
curl http://localhost:11434/api/chat -d '{
  "model": "sven-llama3.1:8b-instruct-q5_K_M",
  "messages": [
    {"role": "user", "content": "Hallo, wie kann ich dir helfen?"}
  ],
}
```

```
"stream": false
}'
```

llama3.1:8b-instruct-q5_K_M

STANDARD

TECHNISCHE DATEN

Hersteller	Meta AI
Veröffentlicht	Juli 2024
Quantisierung	Q5_K_M (5.7 Bit/Gewicht)
Speicherbedarf	~5.3 GB RAM/VRAM
Kontextfenster	128.000 Token
Sprachen	Mehrsprachig (Deutsch sehr gut)
Lizenz	Llama 3.1 Community License (kostenlos)

Was kann dieses Modell?

Das originale Llama 3.1 8B Instruct von Meta ist ein leistungsstarkes Open-Source-Sprachmodell für allgemeine Chat-Aufgaben. Es versteht und antwortet auf Deutsch sehr zuverlässig. "Instruct" bedeutet, es wurde speziell für Anweisungsbefolgung und Dialog optimiert.

API-Aufruf mit Systemprompt:

```
curl http://localhost:11434/api/chat -d '{
  "model": "llama3.1:8b-instruct-q5_K_M",
  "messages": [
    {
      "role": "system",
      "content": "Du bist ein hilfreicher Assistent. Antworte immer auf
    },
    {
      "role": "user",
      "content": "Erkläre mir maschinelles Lernen in 3 Sätzen."
    }
  ]
}'
```

```
}  
],  
"stream": false  
'}
```

Arena Model

WEBUI-FEATURE

Das "Arena Model" ist **kein eigenes KI-Modell**, sondern eine Funktion von Open WebUI. Inspiriert vom LMSYS Chatbot Arena leitet es Anfragen verdeckt an verschiedene Modelle weiter. Man sieht die Antwort, aber nicht welches Modell geantwortet hat – erst nach der Bewertung wird aufgedeckt, welches Modell besser war.

Sinn & Zweck: Objektiver Qualitätsvergleich zwischen Modellen ohne Vorurteile durch den bekannten Modellnamen. Sehr nützlich um zu testen, ob das angepasste `sven-`-Modell besser antwortet als das Standard-Modell.

Nur über die Open WebUI-Oberfläche nutzbar, nicht direkt via API ansprechbar.

Systemprompts – Konzept & Anwendung

Ein **Systemprompt** ist eine versteckte Anweisung, die dem Modell vor der ersten Nutzernachricht mitgegeben wird. Er legt Persönlichkeit, Verhalten, Sprache und Einschränkungen des Modells fest.

Methode 1: Dauerhaft via Modelfile (wie bei sven-llama3.1)

Einmalig erstellt, immer aktiv. Ideal für persönliche Modelle:

```
FROM llama3.1:8b-instruct-q5_K_M

SYSTEM """
Du bist Svens persönlicher KI-Assistent.
Antworte immer auf Deutsch.
Sei präzise, freundlich und professionell.
Wenn du etwas nicht weißt, sage es ehrlich.
"""

PARAMETER temperature 0.7
PARAMETER top_p 0.9
```

```
# Modell erstellen / aktualisieren
ollama create sven-llama3.1:8b-instruct-q5_K_M -f Modelfile
```

Methode 2: Pro Anfrage (für Standard-Modell)

Flexibel, pro Chat-Sitzung anpassbar:

```
{
  "model": "llama3.1:8b-instruct-q5_K_M",
  "messages": [
    {
      "role": "system",
      "content": "Du bist Experte für Ernährungsberatung. Antworte auf Deutsch"
    },
    {
      "role": "user",
```

```
    "content": "Was sollte ich zum Frühstück essen?"  
  }  
]  
}
```

Methode 3: Open WebUI System Prompt

In Open WebUI unter **Einstellungen** → **Modelle** kann pro Modell ein Standard-Systemprompt in der Oberfläche hinterlegt werden – ohne Modelfile-Änderung, bequem über die UI verwaltbar.

⚙️ Technische Details

Quantisierung erklärt (Q5_K_M)

Quantisierung reduziert die Genauigkeit der Modellgewichte, um Speicher zu sparen. Q5_K_M ist die empfohlene Balance für die meisten Nutzer:

FORMAT	BIT/GEWICHT	RAM (8B)	QUALITÄT	EMPFEHLUNG
Q4_K_M	~4.5 Bit	~4.5 GB	Gut	Wenig RAM
Q5_K_M	~5.7 Bit	~5.3 GB	Sehr gut	Empfohlen ✓
Q8_0	~8 Bit	~8.0 GB	Exzellent	Viel RAM nötig
F16	16 Bit	~16 GB	Original	Server/GPU

Ollama API-Endpunkte Übersicht

ENDPUNKT	METHODE	BESCHREIBUNG
<code>/api/tags</code>	GET	Alle installierten Modelle auflisten
<code>/api/chat</code>	POST	Chat-Konversation (mit Verlauf)
<code>/api/generate</code>	POST	Einfache Textvervollständigung
<code>/api/show</code>	POST	Modelldetails & Modelfile anzeigen
<code>/api/pull</code>	POST	Neues Modell herunterladen
<code>/api/create</code>	POST	Modell aus Modelfile erstellen
<code>/v1/chat/completions</code>	POST	OpenAI-kompatibler Endpunkt

Datenschutz-Hinweis: Alle Modelle laufen vollständig lokal auf Svens Rechner. Keine Nutzerdaten, keine Anfragen, keine Antworten verlassen das Gerät. Internet ist nur

zum Herunterladen neuer Modelle erforderlich.

Nützliche Terminal-Befehle

```
# Alle installierten Modelle anzeigen
ollama list

# Systemprompt eines Modells einsehen
ollama show sven-llama3.1:8b-instruct-q5_K_M --modelfile

# Modell direkt im Terminal testen
ollama run sven-llama3.1:8b-instruct-q5_K_M

# Neues Modell herunterladen (Beispiel)
ollama pull mistral:7b

# Laufende Modelle anzeigen
ollama ps
```